Chapter 8: Probabilistic Sequence and Language Models

Part III — Statistical Foundations of Deep Learning & Reinforcement Learning

Omid Safarzadeh

Abstract

A language model is a probability distribution over sequences, and everything it does follows from one identity: the chain rule of probability, which factorises the joint probability of a sentence into a product of next-token conditionals. Training such a model is maximum likelihood on that factorisation — so its loss is cross-entropy and its reported score, perplexity, is the exponentiated cross-entropy of Chapter 5. This chapter builds the autoregressive model from the chain rule, places n-gram and neural models on the same probabilistic footing, and then reads two pieces of the modern stack through a statistical lens: word embeddings, which turn out to factorise a pointwise-mutual-information matrix, and attention, which is kernel-weighted averaging — Nadaraya–Watson regression — in disguise.

Contents

1 The chain rule and the autoregressive model 1

2 Embeddings factorise a PMI matrix 2

3 Attention is kernel smoothing 2

1 The chain rule and the autoregressive model

Theorem 1.1 (Chain rule of probability)**.** For any sequence $x_{1:n} = (x_1, \dots, x_n)$,

$$p(x_{1:n}) = \prod_{t=1}^{n} p(x_t \mid x_{1:t-1}).$$

Proof. Repeated application of the definition of conditional probability gives

$$p(x_{1:n}) = p(x_{1:n-1})\, p(x_n \mid x_{1:n-1}),$$

and unrolling the first factor the same way down to $p(x_1)$ gives the product. □

This exact, assumption-free factorisation is the **autoregressive** model: to model a sequence it suffices to model the next token given the past. An **n-gram** model makes the Markov approximation

$$p(x_t \mid x_{1:t-1}) \approx p(x_t \mid x_{t-n+1:t-1})$$

— condition on the last $n-1$ tokens only — and estimates each conditional by counting. A neural language model keeps the exact conditioning on all of $x_{1:t-1}$ but parameterises $p(x_t \mid x_{1:t-1})$ with a network. Either way the object is the same conditional distribution.

Fitting it is maximum likelihood: maximise $\sum_t \log p(x_t \mid x_{1:t-1})$, which is minimising cross-entropy against the data (Chapter 2). So the loss is the cross-entropy of Chapter 5, and the model's **perplexity** is $e^{\text{cross-entropy}}$ — the effective number of choices it is torn between at each

token. The left panel shows held-out perplexity dropping as the context grows from unigram to trigram on a second-order source, then ticking up for the 4-gram as the counts grow sparse: more context helps only while the data can still estimate it.

2 Embeddings factorise a PMI matrix

Before a token enters a network it is an **embedding**, a dense vector, and the surprising fact is what those vectors encode. **word2vec**'s skip-gram-with-negative-sampling objective, trained on which words co-occur, has an exact interpretation.

Theorem 2.1 (word2vec is implicit matrix factorisation)**.** Skip-gram with negative sampling (with k negatives) is, at its optimum, factorising the matrix of **shifted pointwise mutual information**: word and context vectors satisfy

$$v_w^\top c_c \;=\; \mathrm{PMI}(w,c) - \log k, \qquad \mathrm{PMI}(w,c) = \log \frac{p(w,c)}{p(w)\,p(c)}.$$

The content is that semantics is geometry: PMI is large for words that co-occur more than chance, so factorising it places words that share contexts near one another. The right panel builds a PMI matrix from a toy two-topic corpus and factorises it; the monarchy words and the animal words fall into two separated clusters, with no labels ever supplied. This is the same low-rank, SVD-of-an-association-matrix idea as PCA (Part II) — embeddings are PCA for co-occurrence.

3 Attention is kernel smoothing

The transformer's one new operation is **attention**: a query attends to a set of key–value pairs and returns a weighted average of the values. Written out, with query q, keys k_i, values v_i,

$$\mathrm{attn}(q) = \sum_i \alpha_i\, v_i, \qquad \alpha_i = \frac{\exp(q^\top k_i/\sqrt{d})}{\sum_j \exp(q^\top k_j/\sqrt{d})}.$$

This is a statistical object a century old.

Definition 3.1 (Nadaraya–Watson estimator)**.** The kernel-regression estimate of $\mathbb{E}[v \mid q]$ from pairs (k_i, v_i) is

$$\hat{m}(q) = \frac{\sum_i K(q,k_i)\, v_i}{\sum_i K(q,k_i)},$$

a weighted average of the values with weights given by a similarity kernel K.

Attention *is* the Nadaraya–Watson estimator with the kernel $K(q,k) = \exp(q^\top k/\sqrt{d})$: the softmax normalisation is exactly the denominator $\sum_i K$, and the output is exactly the kernel-weighted mean of the values. The middle panel makes this concrete — attention over scattered key–value points traces the same smooth regression curve a kernel smoother would, with the softmax weights for a query peaking at the nearby keys. Seeing attention this way demystifies it: it is nonparametric regression where the data points are the other tokens, the thing a transformer layer does is smooth each position's representation against the others, and "what to attend to" is just "which keys are similar to this query."

Summary

A language model is a distribution over sequences; the chain rule factorises it into next-token conditionals, so the autoregressive model only needs $p(x_t \mid x_{1:t-1})$ — Markov-truncated (n-

gram) or neural. Fitting it is maximum likelihood, i.e. minimising cross-entropy, and perplexity is $e^{\text{cross-entropy}}$ (Chapter 5). Word embeddings are not arbitrary: word2vec implicitly factorises the shifted-PMI matrix, so semantics becomes geometry (PCA for co-occurrence). Attention is the Nadaraya–Watson kernel estimator with kernel $\exp(q^\top k/\sqrt{d})$ — a kernel-weighted average of values, i.e. nonparametric regression over the other tokens.